BAMAT207 — Probability and Statistics

Project Activity Report

SpamShield

Probabilistic SMS Spam Detection using Bayes' Theorem and Multinomial Naive Bayes

Topic: Probability and Random Variables

Submitted by: Mohammad Asif Shaik

Register No.: 25BDS0107

Faculty: Dhivyaa C R

GitHub: https://github.com/asifzxs/Probability-Stats-NaiveBayes-

Abstract

This project builds a Python system that classifies an SMS message as Spam or Genuine (Ham). Using the UCI SMS Spam Collection (5,574 messages, 5,171 after removing duplicates), we compute class probabilities, conditional probabilities of suspicious words (verified with Bayes' theorem), the expectation and variance of a Bernoulli random variable, and train a Multinomial Naive Bayes classifier. The model achieves 98.07% accuracy on a stratified 20% test set. An interactive web application (SpamShield) lets a user type any SMS and see the prediction, P(Spam | Message), P(Ham | Message) and the word-by-word evidence behind the decision.

1. Objective

To show how dataset analysis, conditional probability, Bayes' theorem, expectation and variance support the development and evaluation of a basic AI classification system.

2. Tools and Libraries

Library	Purpose
pandas	Load, clean and manipulate the dataset
numpy	Probability, expectation and variance
matplotlib / plotly	Class distribution and model result charts
scikit-learn	train_test_split, CountVectorizer, MultinomialNB, metrics
ucimlrepo / urllib	Download the dataset from UCI
re	Whole-word search for suspicious words
streamlit	Interactive web application for the live demo

3. Dataset Handling

Dataset: SMS Spam Collection, UCI Machine Learning Repository (Almeida & Hidalgo, 2011).

Step	Result
Raw messages	5574
Missing values	0
Duplicate messages removed	403
Clean messages	5171
Label encoding	Spam = 1, Ham = 0

4. Dataset Analysis

P(Spam) = 653 / 5171 = 0.1263

P(Ham) = 4518 / 5171 = 0.8737

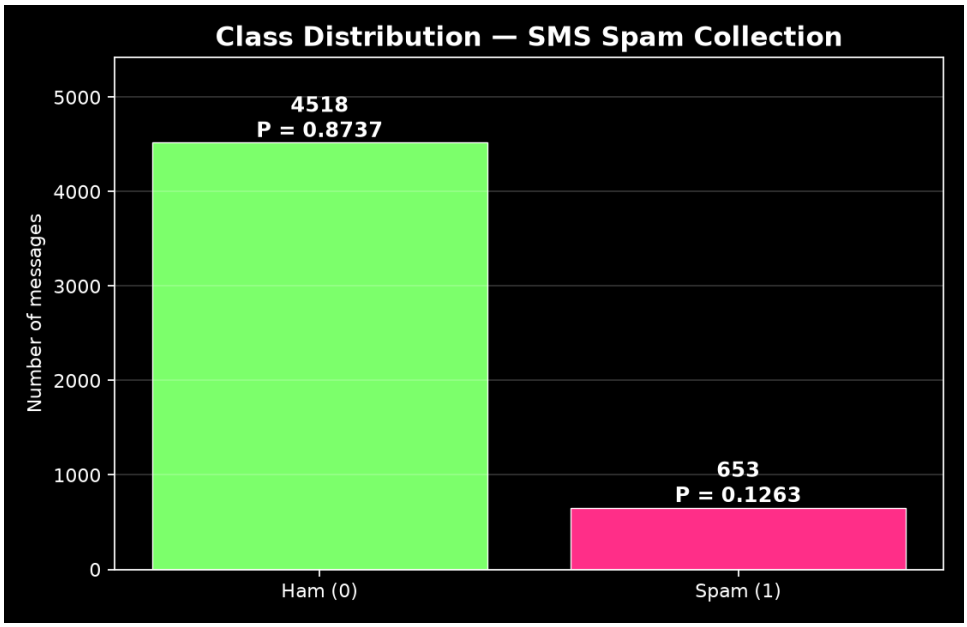


Figure 1: Class distribution of spam and genuine messages

The dataset is imbalanced: only about one message in eight is spam.

5. Word-Based Probability Analysis

Direct: P(Spam | w) = (spam messages containing w) / (all messages containing w)
Bayes: P(Spam | w) = P(w | Spam) × P(Spam) / P(w)

Word	Msgs	Spam	P(w|Spam)	P(w)	Direct	Bayes
free	203	148	0.2266	0.0393	0.7291	0.7291
prize	73	73	0.1118	0.0141	1.0000	1.0000
win	55	49	0.0750	0.0106	0.8909	0.8909
urgent	62	57	0.0873	0.0120	0.9194	0.9194
offer	30	24	0.0368	0.0058	0.8000	0.8000

Worked example (free): P(Spam | Free) = 148/203 = 0.7291. By Bayes: (0.2266 × 0.1263) / 0.0393 = 0.7291. Both methods give the same result.

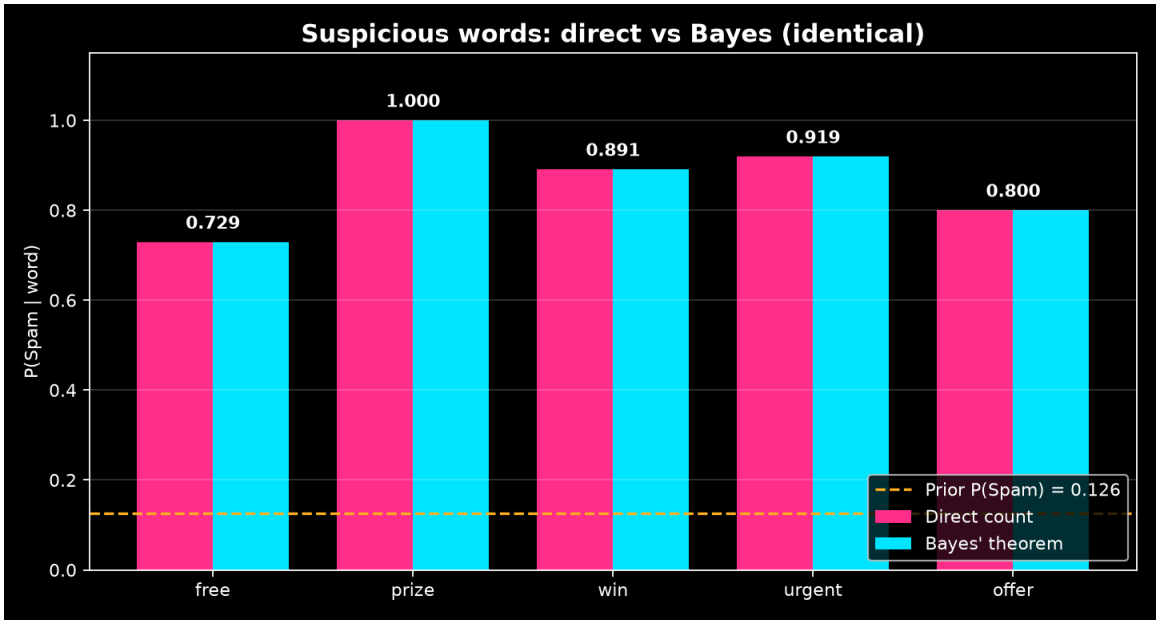


Figure 2: P(Spam | word) by direct counting and by Bayes' theorem

Seeing any of these words raises the spam probability far above the prior of 0.126; every message containing 'prize' in the dataset is spam.

6. Random Variable Analysis

Y = 1 if the message is spam, Y = 0 if it is ham, so Y ~ Bernoulli(p) with p = P(Spam).

$E(Y) = 0\cdot(1-p) + 1\cdot p = p$ $\quad$ $Var(Y) = E(Y^2) - [E(Y)]^2 = p - p^2 = p(1-p)$

Quantity	Value
p = P(Spam)	0.1263
E(Y)	0.1263
Var(Y) = p(1−p)	0.1103
SD(Y)	0.3322
numpy mean / variance of labels	0.1263 / 0.1103

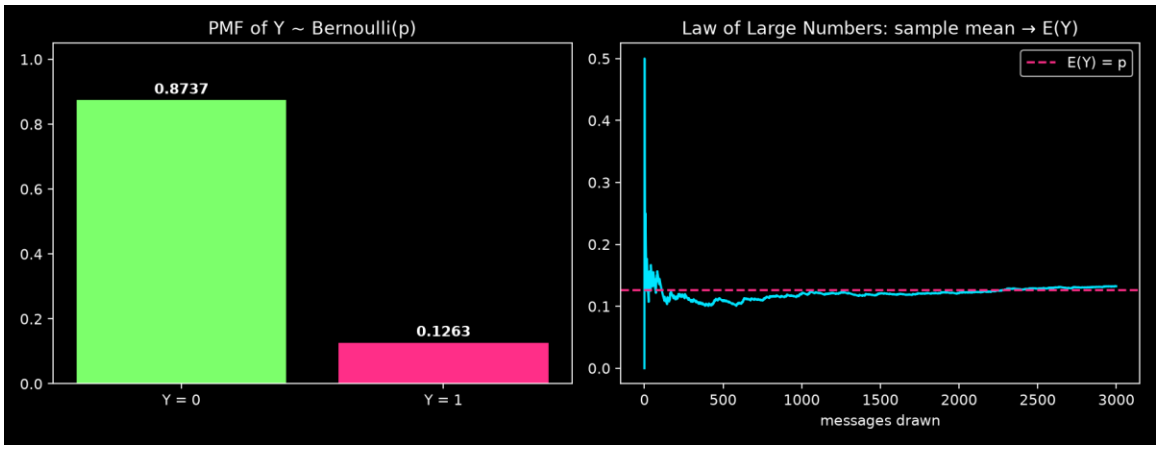


Figure 3: PMF of Y and convergence of the sample mean to E(Y)

7. Data Preparation and Model Building

The data was split 80/20 with stratified sampling (random_state = 42): 4,136 training and 1,035 testing messages, with spam ratios 0.1262 and 0.1266. CountVectorizer, fitted on the training set only, produced a vocabulary of 7,669 words. A Multinomial Naive Bayes classifier (model = MultinomialNB(); model.fit(X_train, y_train)) was trained on these word counts. It computes P(Spam | m) ∝ P(Spam) × Π P(w | Spam)^count(w), assuming words are independent given the class.

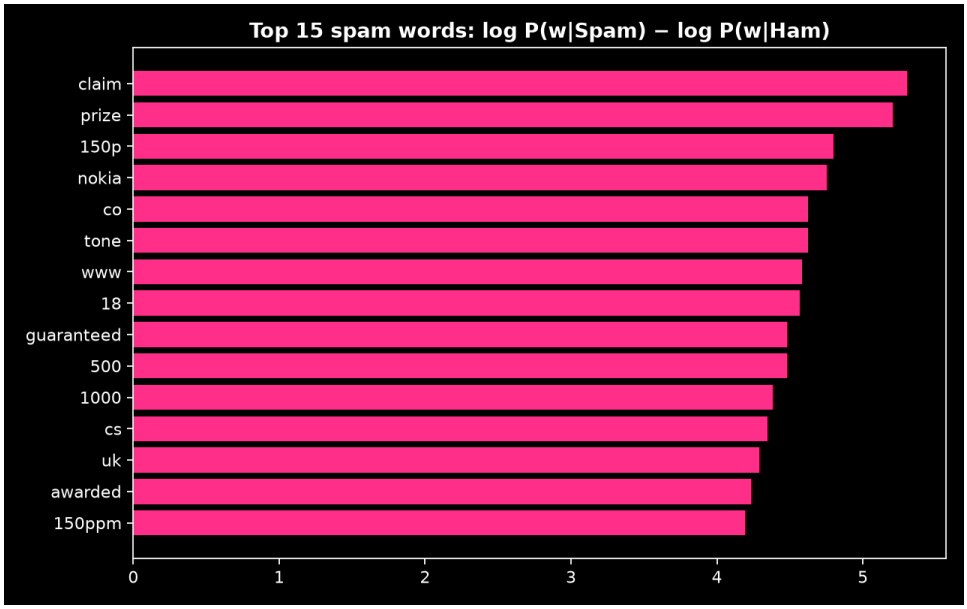


Figure 4: Words the model treats as strongest evidence of spam

8. Model Evaluation

Metric	Value

Accuracy	0.9807
Precision (Spam)	0.9512
Recall (Spam)	0.8931
F1-score (Spam)	0.9213

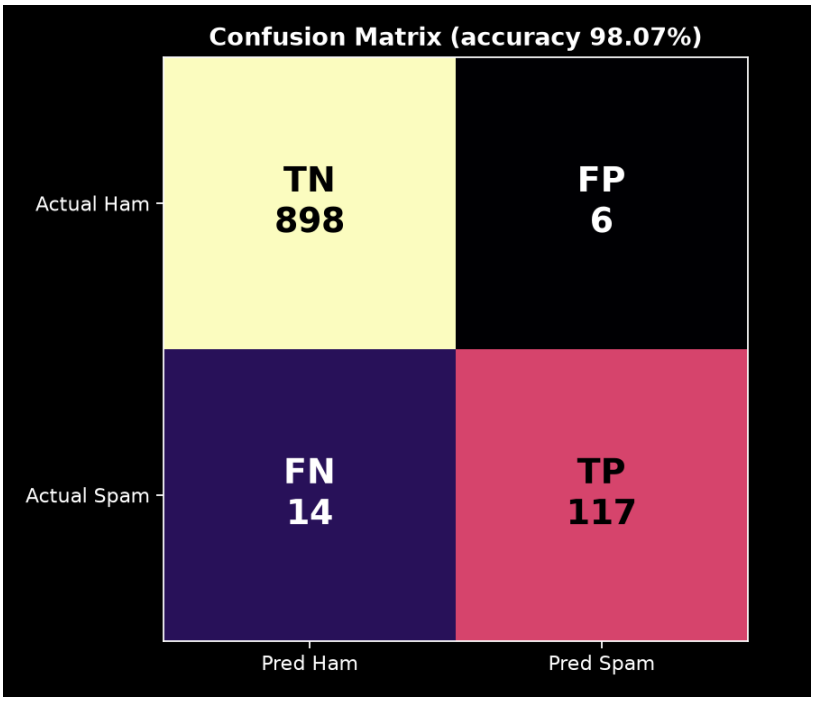


Figure 5: Confusion matrix on the test set

Classification report

	precision	recall	f1-score	support
Ham	0.9846	0.9934	0.9890	904
Spam	0.9512	0.8931	0.9213	131
accuracy			0.9807	1035
macro avg	0.9679	0.9432	0.9551	1035
weighted avg	0.9804	0.9807	0.9804	1035

- Spam messages incorrectly classified as genuine (false negatives): 14
- Genuine messages incorrectly classified as spam (false positives): 6

False positives are the more harmful error for users (a real message is hidden), and the model makes very few of them. Most false negatives are short or conversational spam that contains few typical spam words.

Spam missed by the model (sample)	**P(Spam)**
Check Out Choose Your Babe Videos @	0.370

sms.shsex.netUN fgkslpoPW fgkslpo	
Burger King - Wanna play footy at a top stadium? Get 2 Burger King before 1st Sept and go Large or Super with	0.453
Xmas & New Years Eve tickets are now on sale from the club, during the day from 10am till 8pm, and on Thurs, F	0.005
Would you like to see my XXX pics they are so hot they were nearly banned in the uk!	0.000

9. Real-Time Testing

Message	Prediction	P(Spam|Msg)	P(Ham|Msg)
Congratulations! You have won a free prize. Claim now.	Spam	1.0000	0.0000
URGENT! Your mobile number has won £2000. Call 09061701461 to claim	Spam	1.0000	0.0000
Hey, are we still meeting at 4 for the project demo?	Ham	0.0000	1.0000
Reached home safely, will call you after dinner	Ham	0.0000	1.0000

The notebook accepts typed input through predict_sms(), and the SpamShield web app (streamlit run app.py) adds a live gauge, word-by-word highlighting, a log-odds breakdown of the decision, and an interactive Bayes' theorem calculator.

10. Conclusion

Simple probability gives a strong spam filter. The prior P(Spam) = 0.1263, the conditional probabilities of suspicious words (confirmed by Bayes' theorem) and the Bernoulli model of the label (E(Y) = 0.1263, Var(Y) = 0.1103) all feed directly into the Naive Bayes classifier, which reaches 98.07% accuracy with only 6 genuine messages wrongly flagged out of 904.

Limitations and future work

- Words are treated as independent given the class, which is a simplification of language.
- The model only counts words and does not understand meaning; new spam vocabulary can slip through.

- Predicted probabilities are often very close to 0 or 1 and are not perfectly calibrated.
- Future work: TF-IDF features, character n-grams, and comparison with logistic regression.

References

1. Almeida, T. A., & Hidalgo, J. M. G. (2011). SMS Spam Collection. UCI Machine Learning Repository. https://doi.org/10.24432/C5CC84
2. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12, 2825–2830.
3. Course material, BAMAT207 Probability and Statistics.